Now that we have the table, all we need to do is add a click listener and associated function to populate this dialog, as well as a function to create a div with ID `"modal"` into which we add the loaded HTML code using the `.html()` function:

```
d3.text("resources/modal.html", function(data) {
    d3.select("body").append("div").attr("id", "modal").html(data);
});

teamG.on("click", teamClick);

function teamClick(d) {
    d3.selectAll("td.data").data(d3.values(d))
        .html(function(p) {
            return p
        });
};
```

Creates a new div with an id corresponding to one in our CSS, and populates it with HTML content from modal.html

Selects and updates the td.data elements with the values of the team clicked

The results are immediately apparent when you reload the page. A div with the defined table in modal.html is created, and when you click it, it populates the div with values from the data bound to the element you click (figure 3.19).

We used `d3.text()` in this case because when working with HTML, it can be more convenient to load the raw HTML code like this and drop it into the `.html()` function of a selected element that you've created. If you use `d3.html()`, then you get HTML nodes that allow you to do more sophisticated manipulation, which you'll see now as we work with pregenerated SVG.

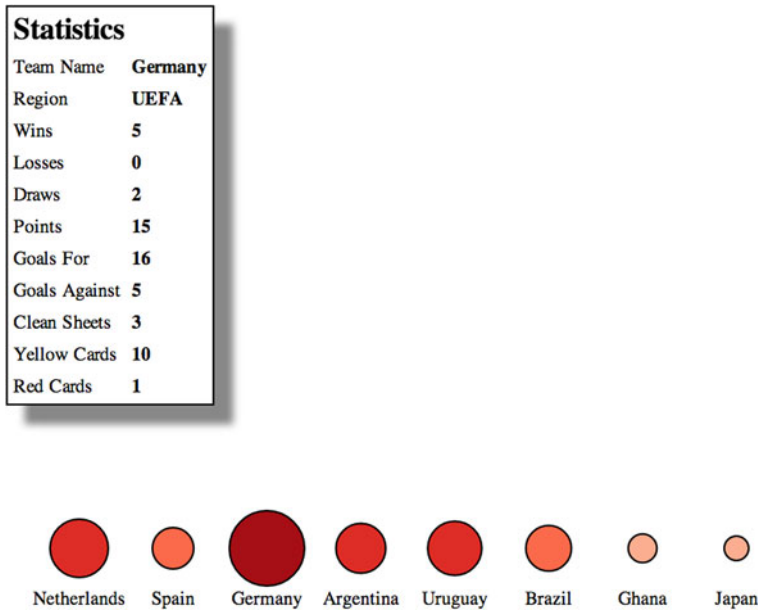


Figure 3.19 The modal dialog is styled based on the defined style in CSS. It's created by loading the HTML data from modal.html and adding it to the content of a newly created div.